

titanic

June 30, 2026

```
[ ]: import pandas as pd

train_data_pd = pd.read_csv('data/train.csv', index_col=0)
train_data_pd.head()
```

0.1 slovník:

Pclass: Třída lístku (1 = První, 2 = Druhá, 3 = Třetí). Funguje to jako zástupný znak pro socioekonomický status.

Name: Jméno pasažéra. (Může obsahovat tituly jako Mr., Mrs., Miss., které samy o sobě nesou zajímavou informaci).

SibSp: Počet sourozenců (Siblings) nebo manželů/manželek (Spouses) daného pasažéra na palubě Titanicu.

Parch: Počet rodičů (Parents) nebo dětí (Children) daného pasažéra na palubě.

Ticket: Číslo lístku.

Fare: Cena lístku (jízdné).

Cabin: Číslo kajuty. (Zde bývá obrovské množství chybějících hodnot).

Embarked: Přístav, kde se pasažér nalodil (C = Cherbourg, Q = Queenstown, S = Southampton)

0.2 text -> number

0.2.1 sex:

male = 0 female = 1

0.2.2 Embarked

C = 0 Q = 1 S = 2

0.2.3 title

MR = 0 MRS = 1 Miss = 2

```
[ ]: # remapping:
train_data_pd['Sex'] = train_data_pd['Sex'].map({'male': 0, 'female': 1})
```

```
train_data_pd['Embarked'] = train_data_pd['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})
```

```
[3]: # title extraction and remapping
train_data_pd['Title'] = train_data_pd['Name'].str.extract(' ([A-Za-z]+)\.', expand=False)

unique_titles = train_data_pd['Title'].unique()
print("Unique titles:", unique_titles)

# 1. Krok: Přemapování synonym (překlepů a cizích jazyků)
train_data_pd['Title'] = train_data_pd['Title'].replace(['Mlle', 'Ms'], 'Miss')
train_data_pd['Title'] = train_data_pd['Title'].replace('Mme', 'Mrs')

# 2. Krok: Vytvoření seznamu všech vzácných titulů
rare_titles = ['Don', 'Rev', 'Dr', 'Major', 'Lady', 'Sir', 'Col', 'Capt', 'the', 'Jonkheer']

# 3. Krok: Hromadné nahrazení vzácných titulů za slovo 'Rare'
train_data_pd['Title'] = train_data_pd['Title'].replace(rare_titles, 'Rare')

# 4. Krok: Kontrola výsledku
print(train_data_pd['Title'].value_counts())

title_mapping = {"Mr": 1, "Miss": 2, "Mrs": 3, "Master": 4, "Rare": 5}
train_data_pd['Title'] = train_data_pd['Title'].map(title_mapping)

# Pro jistotu ošetříme případné NaN hodnoty, kdyby nějaký titul chyběl
train_data_pd['Title'] = train_data_pd['Title'].fillna(0)
train_data_pd.drop(columns=['Name'], inplace=True)
```

```
[4]: train_data_pd.head(20)
```

```
[4]:
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket \
PassengerId							
1	0	3	0	22.0	1	0	A/5 21171
2	1	1	1	38.0	1	0	PC 17599
3	1	3	1	26.0	0	0	STON/O2. 3101282
4	1	1	1	35.0	1	0	113803
5	0	3	0	35.0	0	0	373450
6	0	3	0	NaN	0	0	330877
7	0	1	0	54.0	0	0	17463
8	0	3	0	2.0	3	1	349909
9	1	3	1	27.0	0	2	347742
10	1	2	1	14.0	1	0	237736
11	1	3	1	4.0	1	1	PP 9549
12	1	1	1	58.0	0	0	113783
13	0	3	0	20.0	0	0	A/5. 2151
14	0	3	0	39.0	1	5	347082

15	0	3	1	14.0	0	0	350406
16	1	2	1	55.0	0	0	248706
17	0	3	0	2.0	4	1	382652
18	1	2	0	NaN	0	0	244373
19	0	3	1	31.0	1	0	345763
20	1	3	1	NaN	0	0	2649

	Fare	Cabin	Embarked	Title
PassengerId				
1	7.2500	NaN	0.0	1.0
2	71.2833	C85	1.0	3.0
3	7.9250	NaN	0.0	2.0
4	53.1000	C123	0.0	3.0
5	8.0500	NaN	0.0	1.0
6	8.4583	NaN	2.0	1.0
7	51.8625	E46	0.0	1.0
8	21.0750	NaN	0.0	4.0
9	11.1333	NaN	0.0	3.0
10	30.0708	NaN	1.0	3.0
11	16.7000	G6	0.0	2.0
12	26.5500	C103	0.0	2.0
13	8.0500	NaN	0.0	1.0
14	31.2750	NaN	0.0	1.0
15	7.8542	NaN	0.0	2.0
16	16.0000	NaN	0.0	3.0
17	29.1250	NaN	2.0	4.0
18	13.0000	NaN	0.0	1.0
19	18.0000	NaN	0.0	3.0
20	7.2250	NaN	1.0	3.0

```
[5]: train_data_pd.sort_values(by='Ticket', ascending=True).head(20)
```

```
[5]:
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	\
PassengerId										
505	1	1	1	16.0	0	0	110152	86.5000	B79	
258	1	1	1	30.0	0	0	110152	86.5000	B77	
760	1	1	1	33.0	0	0	110152	86.5000	B77	
263	0	1	0	52.0	1	1	110413	79.6500	E67	
559	1	1	1	39.0	1	1	110413	79.6500	E67	
586	1	1	1	18.0	0	2	110413	79.6500	E68	
111	0	1	0	47.0	0	0	110465	52.0000	C110	
476	0	1	0	NaN	0	0	110465	52.0000	A14	
431	1	1	0	28.0	0	0	110564	26.5500	C52	
367	1	1	1	60.0	1	0	110813	75.2500	D37	
171	0	1	0	61.0	0	0	111240	33.5000	B19	
463	0	1	0	47.0	0	0	111320	38.5000	E63	
524	1	1	1	44.0	0	1	111361	57.9792	B18	

330	1	1	1	16.0	0	1	111361	57.9792	B18
890	1	1	0	26.0	0	0	111369	30.0000	C148
605	1	1	0	35.0	0	0	111426	26.5500	NaN
508	1	1	0	NaN	0	0	111427	26.5500	NaN
188	1	1	0	45.0	0	0	111428	26.5500	NaN
807	0	1	0	39.0	0	0	112050	0.0000	A36
634	0	1	0	NaN	0	0	112052	0.0000	NaN

	Embarked	Title
PassengerId		
505	0.0	2.0
258	0.0	2.0
760	0.0	0.0
263	0.0	1.0
559	0.0	3.0
586	0.0	2.0
111	0.0	1.0
476	0.0	1.0
431	0.0	1.0
367	1.0	3.0
171	0.0	1.0
463	0.0	1.0
524	1.0	3.0
330	1.0	2.0
890	1.0	1.0
605	1.0	1.0
508	0.0	1.0
188	0.0	1.0
807	0.0	1.0
634	0.0	1.0

```
[6]: # 1. Zjistíme, kolikrát se každý lístek v datasetu vyskytuje
pocety_listku = train_data_pd['Ticket'].value_counts()

# 2. Vytvoříme nový sloupec tím, že každý lístek přemapujeme na jeho počet_
    ↪ výskytů
train_data_pd['Group_Size'] = train_data_pd['Ticket'].map(pocety_listku)

# 3. Kontrola výsledku na prvních 5 řádcích
print(train_data_pd[['Ticket', 'Group_Size']].head())

train_data_pd.drop(columns=['Ticket'], inplace=True)
```

	Ticket	Group_Size
PassengerId		
1	A/5 21171	1
2	PC 17599	1

3	STON/O2. 3101282	1
4	113803	2
5	373450	1

```
[7]: # Extrakce prvního písmene pomocí regexu
train_data_pd['Deck'] = train_data_pd['Cabin'].str.extract(r'([A-Za-z])',
↳expand=False)

# 1. Výpis všech unikátních písmen (včetně NaN)
print("Unikátní paluby:", train_data_pd['Deck'].unique())

# 2. Přesný počet jednotlivých písmen v datech
# Parametr dropna=False je zde extrémně důležitý, aby sis zobrazil i počet
↳chybějících hodnot!
print("\nPočty pasažérů na palubách:")
print(train_data_pd['Deck'].value_counts(dropna=False))
```

Unikátní paluby: [nan 'C' 'E' 'G' 'D' 'A' 'B' 'F' 'T']

Počty pasažérů na palubách:

Deck

NaN 687

C 59

B 47

D 33

E 32

A 15

F 13

G 4

T 1

Name: count, dtype: int64

```
[8]: cabin_mapping = {"A": 1, "B": 2, "C": 3, "D": 4, "E": 5, "F": 6, "G": 7, "T": 8}
train_data_pd['Deck'] = train_data_pd['Deck'].map(cabin_mapping)
train_data_pd['Deck'] = train_data_pd['Deck'].fillna(0)

train_data_pd.drop(columns=['Cabin'], inplace=True)
```

```
[9]: train_data_pd.head(20)
```

```
[9]:
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	\
PassengerId									
1	0	3	0	22.0	1	0	7.2500	0.0	
2	1	1	1	38.0	1	0	71.2833	1.0	
3	1	3	1	26.0	0	0	7.9250	0.0	
4	1	1	1	35.0	1	0	53.1000	0.0	
5	0	3	0	35.0	0	0	8.0500	0.0	

6	0	3	0	NaN	0	0	8.4583	2.0
7	0	1	0	54.0	0	0	51.8625	0.0
8	0	3	0	2.0	3	1	21.0750	0.0
9	1	3	1	27.0	0	2	11.1333	0.0
10	1	2	1	14.0	1	0	30.0708	1.0
11	1	3	1	4.0	1	1	16.7000	0.0
12	1	1	1	58.0	0	0	26.5500	0.0
13	0	3	0	20.0	0	0	8.0500	0.0
14	0	3	0	39.0	1	5	31.2750	0.0
15	0	3	1	14.0	0	0	7.8542	0.0
16	1	2	1	55.0	0	0	16.0000	0.0
17	0	3	0	2.0	4	1	29.1250	2.0
18	1	2	0	NaN	0	0	13.0000	0.0
19	0	3	1	31.0	1	0	18.0000	0.0
20	1	3	1	NaN	0	0	7.2250	1.0

PassengerId	Title	Group_Size	Deck
1	1.0	1	0.0
2	3.0	1	3.0
3	2.0	1	0.0
4	3.0	2	3.0
5	1.0	1	0.0
6	1.0	1	0.0
7	1.0	1	5.0
8	4.0	4	0.0
9	3.0	3	0.0
10	3.0	2	0.0
11	2.0	2	7.0
12	2.0	1	3.0
13	1.0	1	0.0
14	1.0	7	0.0
15	2.0	1	0.0
16	3.0	1	0.0
17	4.0	5	0.0
18	1.0	1	0.0
19	3.0	1	0.0
20	3.0	1	0.0

```
[10]: # 1. Přesný počet chybějících (NaN) hodnot ve sloupci Age
chybejici_vek = train_data_pd['Age'].isna().sum()
print(f"Počet chybějících hodnot věku: {chybejici_vek}")

# 2. Procentuální podíl (velmi důležité pro tvé rozhodnutí)
celkovy_pocet_cestujicich = len(train_data_pd)
procento_chybejicich = (chybejici_vek / celkovy_pocet_cestujicich) * 100
print(f"Procento chybějících dat věku: {procento_chybejicich:.2f} %")
```

Počet chybějících hodnot věku: 177
Procento chybějících dat věku: 19.87 %

0.2.4 20% trénovací dat odstranit nedává smysl -> je potřeba odhadnout zbytek.

```
[11]: ## odhad pomocí groupby a medianu
age_median = train_data_pd.groupby('Title')['Age'].median()
print(age_median)
```

```
Title
0.0    33.0
1.0    30.0
2.0    21.0
3.0    35.0
4.0     3.5
5.0    49.0
Name: Age, dtype: float64
```

```
[12]: train_data_pd['Age'] = train_data_pd.groupby('Title')['Age'].transform(lambda x:
    ↪ x.fillna(x.median()))

print("Počet chybějících hodnot po doplnění:", train_data_pd['Age'].isna().
    ↪sum())
```

Počet chybějících hodnot po doplnění: 0

```
[13]: print(train_data_pd.isna().sum())
```

```
Survived    0
Pclass      0
Sex          0
Age         0
SibSp       0
Parch       0
Fare        0
Embarked    2
Title       0
Group_Size  0
Deck        0
dtype: int64
```

```
[14]: train_data_pd = train_data_pd.dropna(subset=['Embarked'])
```

```
[15]: train_data_pd.head(50)
```

```
[15]:
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	\
PassengerId									
1	0	3	0	22.0	1	0	7.2500	0.0	

2	1	1	1	38.0	1	0	71.2833	1.0
3	1	3	1	26.0	0	0	7.9250	0.0
4	1	1	1	35.0	1	0	53.1000	0.0
5	0	3	0	35.0	0	0	8.0500	0.0
6	0	3	0	30.0	0	0	8.4583	2.0
7	0	1	0	54.0	0	0	51.8625	0.0
8	0	3	0	2.0	3	1	21.0750	0.0
9	1	3	1	27.0	0	2	11.1333	0.0
10	1	2	1	14.0	1	0	30.0708	1.0
11	1	3	1	4.0	1	1	16.7000	0.0
12	1	1	1	58.0	0	0	26.5500	0.0
13	0	3	0	20.0	0	0	8.0500	0.0
14	0	3	0	39.0	1	5	31.2750	0.0
15	0	3	1	14.0	0	0	7.8542	0.0
16	1	2	1	55.0	0	0	16.0000	0.0
17	0	3	0	2.0	4	1	29.1250	2.0
18	1	2	0	30.0	0	0	13.0000	0.0
19	0	3	1	31.0	1	0	18.0000	0.0
20	1	3	1	35.0	0	0	7.2250	1.0
21	0	2	0	35.0	0	0	26.0000	0.0
22	1	2	0	34.0	0	0	13.0000	0.0
23	1	3	1	15.0	0	0	8.0292	2.0
24	1	1	0	28.0	0	0	35.5000	0.0
25	0	3	1	8.0	3	1	21.0750	0.0
26	1	3	1	38.0	1	5	31.3875	0.0
27	0	3	0	30.0	0	0	7.2250	1.0
28	0	1	0	19.0	3	2	263.0000	0.0
29	1	3	1	21.0	0	0	7.8792	2.0
30	0	3	0	30.0	0	0	7.8958	0.0
31	0	1	0	40.0	0	0	27.7208	1.0
32	1	1	1	35.0	1	0	146.5208	1.0
33	1	3	1	21.0	0	0	7.7500	2.0
34	0	2	0	66.0	0	0	10.5000	0.0
35	0	1	0	28.0	1	0	82.1708	1.0
36	0	1	0	42.0	1	0	52.0000	0.0
37	1	3	0	30.0	0	0	7.2292	1.0
38	0	3	0	21.0	0	0	8.0500	0.0
39	0	3	1	18.0	2	0	18.0000	0.0
40	1	3	1	14.0	1	0	11.2417	1.0
41	0	3	1	40.0	1	0	9.4750	0.0
42	0	2	1	27.0	1	0	21.0000	0.0
43	0	3	0	30.0	0	0	7.8958	1.0
44	1	2	1	3.0	1	2	41.5792	1.0
45	1	3	1	19.0	0	0	7.8792	2.0
46	0	3	0	30.0	0	0	8.0500	0.0
47	0	3	0	30.0	1	0	15.5000	2.0
48	1	3	1	21.0	0	0	7.7500	2.0

49	0	3	0	30.0	2	0	21.6792	1.0
50	0	3	1	18.0	1	0	17.8000	0.0

PassengerId	Title	Group_Size	Deck
1	1.0	1	0.0
2	3.0	1	3.0
3	2.0	1	0.0
4	3.0	2	3.0
5	1.0	1	0.0
6	1.0	1	0.0
7	1.0	1	5.0
8	4.0	4	0.0
9	3.0	3	0.0
10	3.0	2	0.0
11	2.0	2	7.0
12	2.0	1	3.0
13	1.0	1	0.0
14	1.0	7	0.0
15	2.0	1	0.0
16	3.0	1	0.0
17	4.0	5	0.0
18	1.0	1	0.0
19	3.0	1	0.0
20	3.0	1	0.0
21	1.0	2	0.0
22	1.0	1	4.0
23	2.0	1	0.0
24	1.0	1	1.0
25	2.0	4	0.0
26	3.0	4	0.0
27	1.0	1	0.0
28	1.0	4	3.0
29	2.0	1	0.0
30	1.0	1	0.0
31	5.0	1	0.0
32	3.0	2	2.0
33	2.0	1	0.0
34	1.0	1	0.0
35	1.0	2	0.0
36	1.0	2	0.0
37	1.0	1	0.0
38	1.0	1	0.0
39	2.0	2	0.0
40	2.0	2	0.0
41	3.0	1	0.0
42	3.0	2	0.0

43	1.0	1	0.0
44	2.0	3	0.0
45	2.0	1	0.0
46	1.0	1	0.0
47	1.0	1	0.0
48	2.0	1	0.0
49	1.0	1	0.0
50	3.0	2	0.0

```
[16]: import matplotlib.pyplot as plt
import seaborn as sns

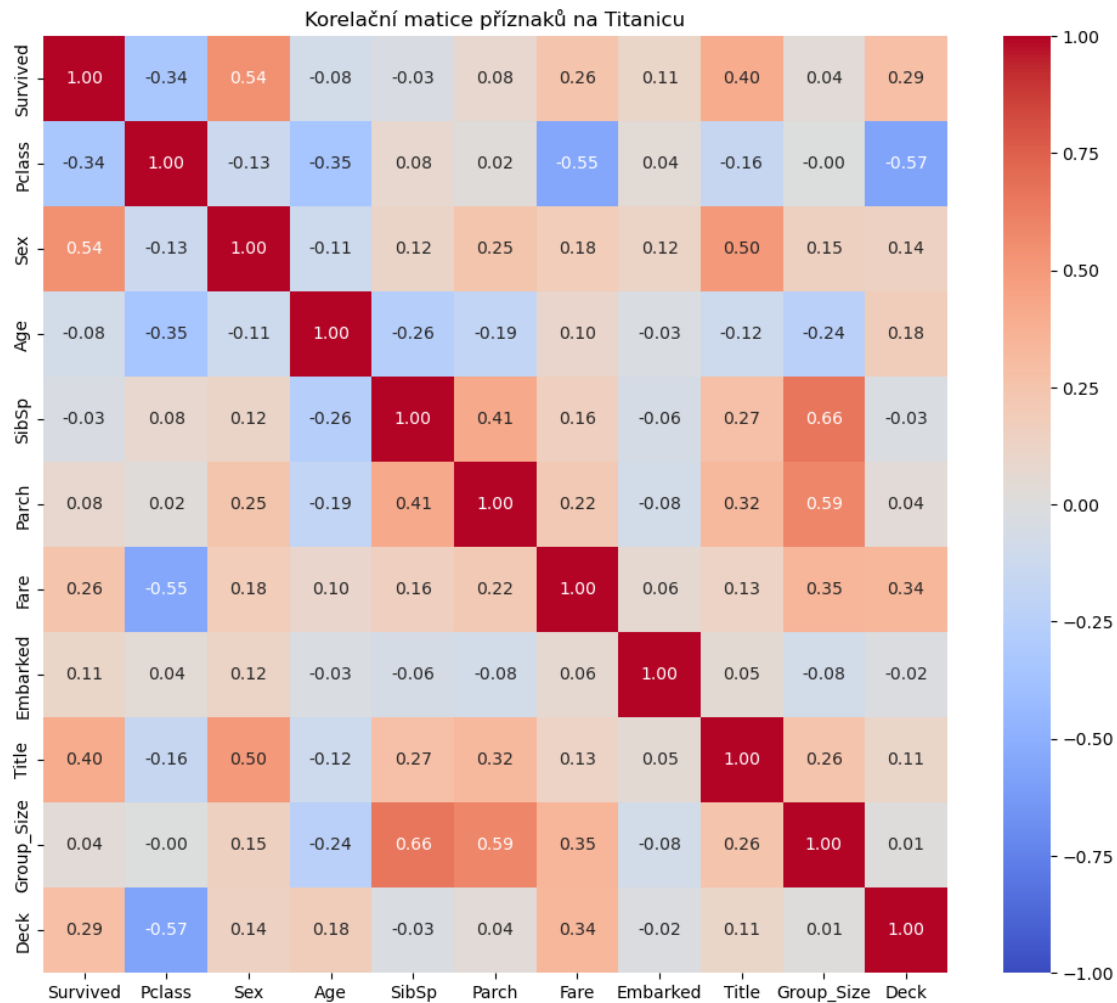
číselná_data = train_data_pd.select_dtypes(include=['int64', 'float64'])

# 2. Samotný výpočet korelační matice (Pearsonův koeficient)
korelacni_matice = číselná_data.corr()

# 3. Vizualizace pomocí teplotní mapy
plt.figure(figsize=(12, 10)) # Nastavení velikosti grafu

# Vykreslení heatmapy:
# annot=True zobrazí čísla, cmap nastaví barvy (od modré po červenou),
# fmt=".2f" zaokrouhlí na 2 desetinná místa
sns.heatmap(korelacni_matice, annot=True, cmap='coolwarm', fmt=".2f", vmin=-1, v
↪max=1)

plt.title('Korelační matice příznaků na Titanicu')
plt.show()
```



```
[17]: # - SibSp a Parch jsou silně korelované s Group_Size, takže je můžeme odstranit
train_data_pd.drop(columns=['SibSp', 'Parch'], inplace=True)
```

```
[18]: ## load test data:
import pandas as pd

def preprocess_titanic_data(df, is_train=True, age_medians=None):
    # Vytvoříme kopii, abychom nepřepisovali původní data v paměti
    df_clean = df.copy()

    # 1. Sex: Přemapování textu na čísla [cite: 26]
    df_clean['Sex'] = df_clean['Sex'].map({'male': 0, 'female': 1})

    # 2. Embarked: Přemapování a ošetření chybějících hodnot [cite: 26]
    df_clean['Embarked'] = df_clean['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})
    if is_train:
```

```

    # V trénovacím setu mažeme 2 řádky bez přístavu
    df_clean = df_clean.dropna(subset=['Embarked'])
else:
    # V testovacím setu doplníme nejčastější přístav (S = 0)
    df_clean['Embarked'] = df_clean['Embarked'].fillna(0)

# 3. Title: Extrakce a sjednocení titulů [cite: 28, 34, 35, 39, 42]
df_clean['Title'] = df_clean['Name'].str.extract(r' ([A-Za-z+])\.', expand=False)
df_clean['Title'] = df_clean['Title'].replace(['Mlle', 'Ms'], 'Miss')
df_clean['Title'] = df_clean['Title'].replace('Mme', 'Mrs')

# Do vzácných titulů přidán 'Dona', který se vyskytuje pouze v testovacím setu
rare_titles = ['Don', 'Rev', 'Dr', 'Major', 'Lady', 'Sir', 'Col', 'Capt', 'the', 'Jonkheer', 'Countess', 'Dona']
df_clean['Title'] = df_clean['Title'].replace(rare_titles, 'Rare')

title_mapping = {"Mr": 1, "Miss": 2, "Mrs": 3, "Master": 4, "Rare": 5}
df_clean['Title'] = df_clean['Title'].map(title_mapping).fillna(0) # Ošetření případných chybějících [cite: 44]
df_clean.drop(columns=['Name'], inplace=True) # [cite: 45]

# 4. Group Size: Velikost skupiny z Ticketu [cite: 139, 141]
pocety_listku = df_clean['Ticket'].value_counts()
df_clean['Group_Size'] = df_clean['Ticket'].map(pocety_listku)
df_clean.drop(columns=['Ticket'], inplace=True) # [cite: 144]

# 5. Deck: Extrakce paluby z kajuty [cite: 147, 179]
df_clean['Deck'] = df_clean['Cabin'].str.extract(r'([A-Za-z])', expand=False)
cabin_mapping = {"A": 1, "B": 2, "C": 3, "D": 4, "E": 5, "F": 6, "G": 7, "T": 8}
df_clean['Deck'] = df_clean['Deck'].map(cabin_mapping).fillna(0) # [cite: 181]
df_clean.drop(columns=['Cabin'], inplace=True) # [cite: 182]

# 6. Age: Doplnění věku s ochranou proti úniku dat
if is_train:
    # Vypočítáme si mediány a uložíme si je pro pozdější vrácení
    age_medians = df_clean.groupby('Title')['Age'].median().to_dict()
    df_clean['Age'] = df_clean['Age'].fillna(df_clean['Title'].map(age_medians))
else:
    # Pokud jsme v testu, použijeme slovník mediánů z tréninku

```

```

        df_clean['Age'] = df_clean['Age'].fillna(df_clean['Title'].
↪map(age_medians))

# 7. Odstranění zbytečných příznaků [cite: 356]
df_clean.drop(columns=['SibSp', 'Parch'], inplace=True, errors='ignore')

# 8. Fare: Ošetření chybějící ceny (častý problém testovacího datasetu)
if not is_train:
    df_clean['Fare'] = df_clean['Fare'].fillna(df_clean['Fare'].median())

# Vrátíme zpracovaná data (a v případě tréninku i vypočtené mediány věku)
if is_train:
    return df_clean, age_medians
return df_clean

```

```

[19]: # Načtení hrubých dat
train_raw = pd.read_csv('data/train.csv', index_col=0)
test_raw = pd.read_csv('data/test.csv', index_col=0)

# 1. Zpracování trénovacích dat (získáme vyčištěný DataFrame a mediány věku)
train_clean, learned_age_medians = preprocess_titanic_data(train_raw, ↪
↪is_train=True)

# 2. Zpracování testovacích dat (předáme funkci naučené mediány z tréninku)
test_clean = preprocess_titanic_data(test_raw, is_train=False, ↪
↪age_medians=learned_age_medians)

```

0.3 KNN

```

[20]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler

# =====
# 2. ZPRACOVÁNÍ DAT
# =====
print("Načítám a čistím data...")
# Nastavení index_col=0 zajistí, že se PassengerId stane indexem a nebude plést ↪
↪model
train_raw = pd.read_csv('data/train.csv', index_col=0)

# Aplikace čistící funkce
train_clean, learned_age_medians = preprocess_titanic_data(train_raw, ↪
↪is_train=True)

```

```

# Rozdělení na příznaky (X) a cílovou proměnnou (y)
y_train = train_clean['Survived']
X_train = train_clean.drop(columns=['Survived'])

# =====
# 3. ŠKÁLOVÁNÍ DAT (Pro KNN naprosto nezbytné)
# =====
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# =====
# 4. HLEDÁNÍ IDEÁLNÍHO K PRO KNN
# =====
print("Trénuji KNN a hledám optimální počet sousedů (K)...")
prumerne_presnosti = []
hodnoty_k = range(1, 101) # Vyzkoušíme K od 1 do 30

for k in hodnoty_k:
    knn = KNeighborsClassifier(n_neighbors=k)
    # 5-fold křížová validace
    skore = cross_val_score(knn, X_train_scaled, y_train, cv=5,
        ↪scoring='accuracy')
    prumerne_presnosti.append(skore.mean())

# Zjištění nejlepšího výsledku
nejlepsi_skore = max(prumerne_presnosti)
nejlepsi_k = hodnoty_k[prumerne_presnosti.index(nejlepsi_skore)]

print(f"Hotovo! Nejlepší přesnosti {nejlepsi_skore*100:.2f} % bylo dosaženo s K_
    ↪ {nejlepsi_k}")

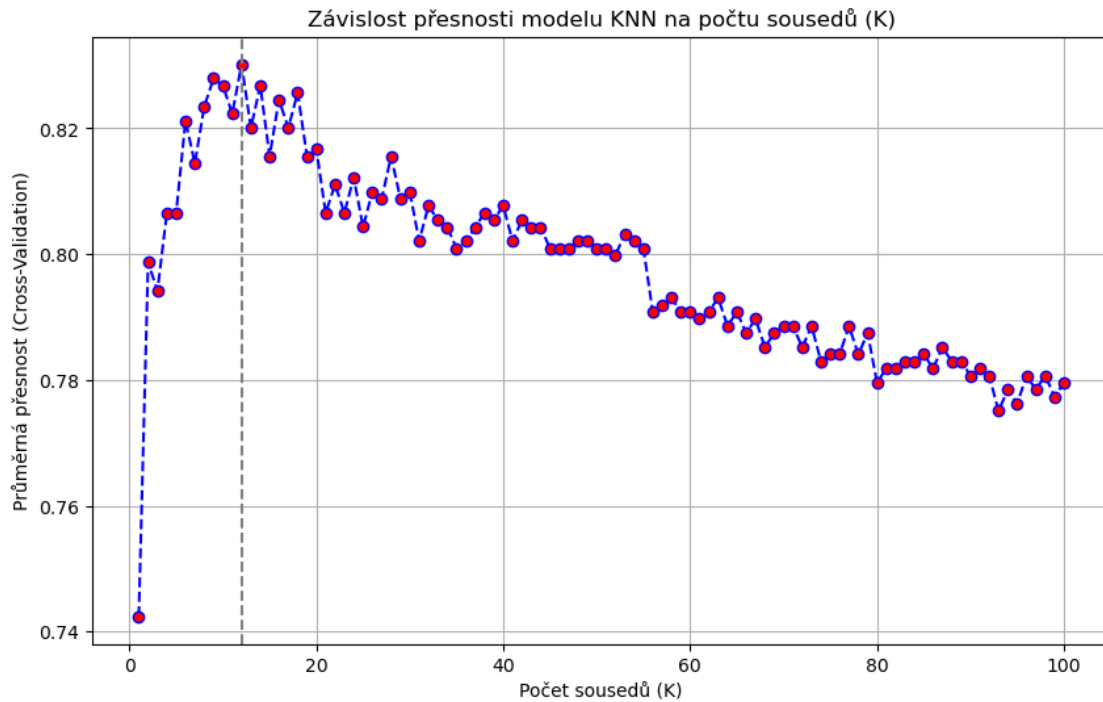
# =====
# 5. VYKRESLENÍ GRAFU
# =====
plt.figure(figsize=(10, 6))
plt.plot(hodnoty_k, prumerne_presnosti, marker='o', linestyle='dashed',
    ↪color='blue', markerfacecolor='red')
plt.title('Závislost přesnosti modelu KNN na počtu sousedů (K)')
plt.xlabel('Počet sousedů (K)')
plt.ylabel('Průměrná přesnost (Cross-Validation)')
plt.axvline(x=nejlepsi_k, color='grey', linestyle='--') # Zvýrazníme nejlepší K
plt.grid(True)
plt.show()

```

Načítám a čistím data...

Trénuji KNN a hledám optimální počet sousedů (K)...

Hotovo! Nejlepší přesnosti 83.02 % bylo dosaženo s $K = 12$



0.3.1 Neuronka

```
[28]: import torch
import torch.nn as nn
import pytorch_lightning as pl
import torchmetrics

class TitanicClassifier(pl.LightningModule):
    def __init__(self, input_dim, hidden_dim=32, learning_rate=1e-3):
        super().__init__()
        # Uložení hyperparametrů pro Lightning
        self.save_hyperparameters()

        # TOTO TI CHYBĚLO:
        self.learning_rate = learning_rate

        self.model = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim // 2), nn.ReLU(),
            nn.Linear(hidden_dim // 2, 1)
        )
        self.criterion = nn.BCEWithLogitsLoss()
```

```

        # Inicializace historie pro grafy
        self.history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

        self.train_acc = torchmetrics.Accuracy(task="binary")
        self.val_acc = torchmetrics.Accuracy(task="binary")

    def forward(self, x):
        return self.model(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        logits = self(x).squeeze()
        loss = self.criterion(logits, y.float())
        acc = self.train_acc(torch.sigmoid(logits), y.int())
        self.log_dict({'train_loss': loss, 'train_acc': acc}, prog_bar=True)
        return loss

    def validation_step(self, batch, batch_idx):
        x, y = batch
        logits = self(x).squeeze()
        loss = self.criterion(logits, y.float())
        acc = self.val_acc(torch.sigmoid(logits), y.int())
        self.log_dict({'val_loss': loss, 'val_acc': acc}, prog_bar=True)
        return loss

    # Přidání ukládání do historie po skončení epochy
    def on_train_epoch_end(self):
        # trainer.callback_metrics obsahuje poslední logované hodnoty
        if 'train_loss' in self.trainer.callback_metrics:
            self.history['train_loss'].append(self.trainer.
callback_metrics['train_loss'].item())

    def on_validation_epoch_end(self):
        # Kontrola, zda máme výsledky z validace
        if 'val_loss' in self.trainer.callback_metrics:
            self.history['val_loss'].append(self.trainer.
callback_metrics['val_loss'].item())
            self.history['val_acc'].append(self.trainer.
callback_metrics['val_acc'].item())

    def configure_optimizers(self):
        # Teď už 'self.learning_rate' existuje
        return torch.optim.Adam(self.parameters(), lr=self.learning_rate)

```



```

[29]: import torch
import torch.nn as nn
import torch.optim as optim
import pytorch_lightning as pl
import pandas as pd
import matplotlib.pyplot as plt
from torch.utils.data import TensorDataset, DataLoader, random_split
import torchmetrics

# 2. PŘÍPRAVA DAT
# Předpokládáme, že máš X_train_scaled a y_train z předchozího kroku
full_dataset = TensorDataset(torch.tensor(X_train_scaled, dtype=torch.float32),
                              torch.tensor(y_train.values, dtype=torch.float32))

# Rozdělení na 90% train / 10% val
val_size = int(0.1 * len(full_dataset))
train_ds, val_ds = random_split(full_dataset, [len(full_dataset) - val_size,
↪val_size])

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=32)

# 3. TRÉNINK
model = TitanicClassifier(input_dim=X_train_scaled.shape[1])
trainer = pl.Trainer(max_epochs=50, accelerator="cpu",
↪enable_model_summary=True)
trainer.fit(model, train_loader, val_loader)

# 4. VIZUALIZACE GRAFŮ
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(model.history['train_loss'], label='Train Loss')
plt.plot(model.history['val_loss'], label='Val Loss')
plt.title('Loss v čase')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(model.history['val_acc'], label='Val Accuracy', color='orange')
plt.title('Validační přesnost')
plt.legend()
plt.show()

# 5. INFERENCE NA TESTOVACÍ SADĚ
X_test_scaled = scaler.transform(X_test)
tensor_X_test = torch.tensor(X_test_scaled, dtype=torch.float32)

model.eval()
with torch.no_grad():

```

```

        predictions = (torch.sigmoid(model(tensor_X_test)).squeeze() >= 0.5).int().
        ↪numpy()

# ULOŽENÍ
output = pd.DataFrame({'PassengerId': test_clean.index if 'PassengerId' not in
        ↪test_clean.columns else test_clean['PassengerId'],
                        'Survived': predictions})
output.to_csv('submission_pytorch.csv', index=False)
print(" Hotovo! Grafy vykresleny, soubor uložen.")

```

GPU available: False, used: False

TPU available: False, using: 0 TPU cores

Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Tip: For seamless cloud uploads and versioning, try installing [litmodels](https://pypi.org/project/litmodels/) to enable LitModelCheckpoint, which syncs automatically with the Lightning model registry.

	Name	Type	Params	Mode	FLOPs
0	model	Sequential	833	train	0
1	criterion	BCEWithLogitsLoss	0	train	0
2	train_acc	BinaryAccuracy	0	train	0
3	val_acc	BinaryAccuracy	0	train	0

Trainable params: 833

Non-trainable params: 0

Total params: 833

Total estimated model params size (MB): 0.003

Modules in train mode: 9

Modules in eval mode: 0

Total FLOPs: 0

Output()

```

/home/luky/.local/lib/python3.10/site-packages/pytorch_lightning/trainer/
    ↪connectors/data_connector.py:434: The
'val_dataloader' does not have many workers which may be a bottleneck. Consider
    ↪increasing the value of the
`num_workers` argument` to `num_workers=7` in the `DataLoader` to improve
    ↪performance.

```

```

/home/luky/.local/lib/python3.10/site-packages/pytorch_lightning/loops/fit_loop.
    ↪py:321: The number of training

```

batches (26) is smaller than the logging interval Trainer(log_every_n_steps=50).
↳ Set a lower value for
log_every_n_steps if you want to see logs for the training epoch.

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[29], line 24  
    22 model = TitanicClassifier(input_dim=X_train_scaled.shape[1])  
    23 trainer = pl.Trainer(max_epochs=50, accelerator="cpu",  
↳ enable_model_summary=True)  
--> 24 trainer.fit(model, train_loader, val_loader)  
    26 # 4. VIZUALIZACE GRAFŮ  
    27 plt.figure(figsize=(12, 5))  
  
File ~/.local/lib/python3.10/site-packages/pytorch_lightning/trainer/trainer.py  
↳ 584, in Trainer.fit(self, model, train_data loaders, val_data loaders,  
↳ datamodule, ckpt_path, weights_only)  
    582 self.training = True  
    583 self.should_stop = False  
--> 584 call._call_and_handle_interrupt(  
    585     self,  
    586     self._fit_impl,  
    587     model,  
    588     train_data loaders,  
    589     val_data loaders,  
    590     datamodule,  
    591     ckpt_path,  
    592     weights_only,  
    593 )  
  
File ~/.local/lib/python3.10/site-packages/pytorch_lightning/trainer/call.py:49  
↳ in _call_and_handle_interrupt(trainer, trainer_fn, *args, **kwargs)  
    47     if trainer.strategy.launcher is not None:  
    48         return trainer.strategy.launcher.launch(trainer_fn, *args,  
↳ trainer=trainer, **kwargs)  
--> 49     return trainer_fn(*args, **kwargs)  
    51 except _TunerExitException:  
    52     _call_teardown_hook(trainer)  
  
File ~/.local/lib/python3.10/site-packages/pytorch_lightning/trainer/trainer.py  
↳ 630, in Trainer._fit_impl(self, model, train_data loaders, val_data loaders,  
↳ datamodule, ckpt_path, weights_only)  
    623     download_model_from_registry(ckpt_path, self)  
    624 ckpt_path = self.checkpoint_connector._select_ckpt_path(  
    625     self.state.fn,
```

```

626     ckpt_path,
627     model_provided=True,
628     model_connected=self.lightning_module is not None,
629 )
--> 630 self._run(model, ckpt_path=ckpt_path, weights_only=weights_only)
632 assert self.state.stopped
633 self.training = False

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/trainer/trainer.py
↪1079, in Trainer._run(self, model, ckpt_path, weights_only)
1074 self._signal_connector.register_signal_handlers()
1076 # -----
1077 # RUN THE TRAINER
1078 # -----
-> 1079 results = self._run_stage()
1081 # -----
1082 # POST-Training CLEAN UP
1083 # -----
1084 log.debug(f"{self.__class__.__name__}: trainer tearing down")

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/trainer/trainer.py
↪1123, in Trainer._run_stage(self)
1121     self._run_sanity_check()
1122     with torch.autograd.set_detect_anomaly(self._detect_anomaly):
-> 1123         self.fit_loop.run()
1124     return None
1125 raise RuntimeError(f"Unexpected state {self.state}")

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/loops/fit_loop.py:
↪217, in _FitLoop.run(self)
215 try:
216     self.on_advance_start()
--> 217     self.advance()
218     self.on_advance_end()
219 except StopIteration:

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/loops/fit_loop.py:
↪469, in _FitLoop.advance(self)
467 with self.trainer.profiler.profile("run_training_epoch"):
468     assert self._data_fetcher is not None
--> 469     self.epoch_loop.run(self._data_fetcher)

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/loops/
↪training_epoch_loop.py:153, in _TrainingEpochLoop.run(self, data_fetcher)
151 while not self.done:
152     try:
--> 153         self.advance(data_fetcher)
154         self.on_advance_end(data_fetcher)

```

```

155     except StopIteration:

File ~/local/lib/python3.10/site-packages/pytorch_lightning/loops/
↳training_epoch_loop.py:352, in _TrainingEpochLoop.advance(self, data_fetcher)
    349 with trainer.profiler.profile("run_training_batch"):
    350     if trainer.lightning_module.automatic_optimization:
    351         # in automatic optimization, there can only be one optimizer
--> 352         batch_output =
↳self.automatic_optimization.run(trainer.optimizers[0], batch_idx, kwargs)
    353     else:
    354         batch_output = self.manual_optimization.run(kwargs)

File ~/local/lib/python3.10/site-packages/pytorch_lightning/loops/optimization
↳automatic.py:192, in _AutomaticOptimization.run(self, optimizer, batch_idx,
↳kwargs)
    185         closure()
    187 # -----
    188 # BACKWARD PASS
    189 # -----
    190 # gradient update with accumulated gradients
    191 else:
--> 192     self._optimizer_step(batch_idx, closure)
    194 result = closure.consume_result()
    195 if result.loss is None:

File ~/local/lib/python3.10/site-packages/pytorch_lightning/loops/optimization
↳automatic.py:270, in _AutomaticOptimization._optimizer_step(self, batch_idx,
↳train_step_and_backward_closure)
    267     self.optim_progress.optimizer.step.increment_ready()
    269 # model hook
--> 270 call._call_lightning_module_hook(
    271     trainer,
    272     "optimizer_step",
    273     trainer.current_epoch,
    274     batch_idx,
    275     optimizer,
    276     train_step_and_backward_closure,
    277 )
    279 if not should_accumulate:
    280     self.optim_progress.optimizer.step.increment_completed()

File ~/local/lib/python3.10/site-packages/pytorch_lightning/trainer/call.py:
↳177, in _call_lightning_module_hook(trainer, hook_name, pl_module, *args,
↳**kwargs)
    174 pl_module._current_fx_name = hook_name
    176 with trainer.profiler.profile(f"[LightningModule]{pl_module.__class__}
↳_{__name__}.{hook_name}"):
--> 177     output = fn(*args, **kwargs)

```

```

179 # restore current_fx when nested context
180 pl_module._current_fx_name = prev_fx_name

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/core/module.py:
↳ 1368, in LightningModule.optimizer_step(self, epoch, batch_idx, optimizer,
↳ optimizer_closure)
    1337 def optimizer_step(
    1338     self,
    1339     epoch: int,
    (...)
    1342     optimizer_closure: Optional[Callable[[], Any]] = None,
    1343 ) -> None:
    1344     r"""Override this method to adjust the default way the :class:
↳ ~pytorch_lightning.trainer.trainer.Trainer` calls
    1345     the optimizer.
    1346
    (...)
    1366
    1367     """
-> 1368     optimizer.step(closure=optimizer_closure)

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/core/optimizer.py:
↳ 154, in LightningOptimizer.step(self, closure, **kwargs)
    151     raise MisconfigurationException("When `optimizer.step(closure)` is
↳ called, the closure should be callable")
    153 assert self._strategy is not None
-> 154 step_output =
↳ self._strategy.optimizer_step(self._optimizer, closure, **kwargs)
    156 self._on_after_step()
    158 return step_output

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/strategies/strateg .
↳ py:239, in Strategy.optimizer_step(self, optimizer, closure, model, **kwargs)
    237 # TODO(fabric): remove assertion once strategy's optimizer_step typing
↳ is fixed
    238 assert isinstance(model, pl.LightningModule)
-> 239 return
↳ self.precision_plugin.optimizer_step(optimizer, model=model, closure=closure, **kwargs)

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/plugins/precision/
↳ precision.py:123, in Precision.optimizer_step(self, optimizer, model, closure
↳ **kwargs)
    121 """Hook to run the optimizer step."""
    122 closure = partial(self._wrap_closure, model, optimizer, closure)
-> 123 return optimizer.step(closure=closure, **kwargs)

File ~/.local/lib/python3.10/site-packages/torch/optim/optimizer.py:535, in
↳ Optimizer.profile_hook_step.<locals>.wrapper(*args, **kwargs)

```

```

530         raise RuntimeError(
531             f"{func} must return None or a tuple of (new_args,
↳new_kwargs), but got {result}."
532         )
534 # pyrefly: ignore [invalid-param-spec]
--> 535 out = func(*args, **kwargs)
536 self._optimizer_step_code()
538 # call optimizer step post hooks

```

```

File ~/.local/lib/python3.10/site-packages/torch/optim/optimizer.py:80, in
↳_use_grad_for_differentiable.<locals>._use_grad(*args, **kwargs)
    78     torch.set_grad_enabled(self.defaults["differentiable"])
    79     torch._dynamo.graph_break()
--> 80     ret = func(*args, **kwargs)
    81 finally:
    82     torch._dynamo.graph_break()

```

```

File ~/.local/lib/python3.10/site-packages/torch/optim/adam.py:227, in Adam.
↳step(self, closure)
    225 if closure is not None:
    226     with torch.enable_grad():
--> 227         loss = closure()
    229 for group in self.param_groups:
    230     params_with_grad: list[Tensor] = []

```

```

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/plugins/precision/
↳precision.py:109, in Precision._wrap_closure(self, model, optimizer, closure)
    96 def _wrap_closure(
    97     self,
    98     model: "pl.LightningModule",
    99     optimizer: Steppable,
   100     closure: Callable[[], Any],
   101 ) -> Any:
   102     """This double-closure allows makes sure the ``closure`` is execute
↳before the ``on_before_optimizer_step``
   103     hook is called.
   104
   105     (...)
   107
   108     """
--> 109     closure_result = closure()
   110     self._after_closure(model, optimizer)
   111     return closure_result

```

```

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/loops/optimization
↳automatic.py:146, in Closure.__call__(self, *args, **kwargs)
    144 @override
    145 def __call__(self, *args: Any, **kwargs: Any) -> Optional[Tensor]:

```

```
--> 146     self._result = self.closure(*args, **kwargs)
      147     return self._result.loss
```

File ~/.local/lib/python3.10/site-packages/torch/utils/_contextlib.py:124, in

```
↳ context_decorator.<locals>.decorate_context(*args, **kwargs)
      120 @functools.wraps(func)
      121 def decorate_context(*args, **kwargs):
      122     # pyrefly: ignore [bad-context-manager]
      123     with ctx_factory():
--> 124         return func(*args, **kwargs)
```

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/loops/optimization

```
↳ automatic.py:131, in Closure.closure(self, *args, **kwargs)
      128 @override
      129 @torch.enable_grad()
      130 def closure(self, *args: Any, **kwargs: Any) -> ClosureResult:
--> 131     step_output = self._step_fn()
      132     if step_output.closure_loss is None:
      133         self.warning_cache.warn("`training_step` returned `None`. If
↳ this was on purpose, ignore this warning...")
```

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/loops/optimization

```
↳ automatic.py:319, in _AutomaticOptimization._training_step(self, kwargs)
      308 """Performs the actual train step with the tied hooks.
      309
      310 Args:
      311 (...)
      312
      313 """
      314 trainer = self.trainer
--> 319 training_step_output =
↳ call._call_strategy_hook(trainer, "training_step", *kwargs.values())
      320 self.trainer.strategy.post_training_step() # unused hook - call anyway
↳ for backward compatibility
      321 if training_step_output is None and trainer.world_size > 1:
```

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/trainer/call.py:

```
↳ 329, in _call_strategy_hook(trainer, hook_name, *args, **kwargs)
      326     return None
      327 with trainer.profiler.profile(f"[Strategy]{trainer.strategy.__class__}
↳ __name__}.{hook_name}"):
--> 329     output = fn(*args, **kwargs)
      330 # restore current_fx when nested context
      331 pl_module._current_fx_name = prev_fx_name
```

File ~/.local/lib/python3.10/site-packages/pytorch_lightning/strategies/strateg

```
↳ py:391, in Strategy.training_step(self, *args, **kwargs)
      389 if self.model != self.lightning_module:
```



```

    390     return self._forward_redirection(self.model, self.lightning_module,
    ↪ "training_step", *args, **kwargs)
--> 391 return self.lightning_module.training_step(*args, **kwargs)

```

```

Cell In[28], line 33, in TitanicClassifier.training_step(self, batch, batch_idx
    31 x, y = batch
    32 logits = self(x).squeeze()
--> 33 loss = self.criterion(logits, y.float())
    34 acc = self.train_acc(torch.sigmoid(logits), y.int())
    35 self.log_dict({'train_loss': loss, 'train_acc': acc}, prog_bar=True)

```

```

File ~/.local/lib/python3.10/site-packages/torch/nn/modules/module.py:1778, in
    ↪ Module._wrapped_call_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type:
    ↪ ignore[misc]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

```

```

File ~/.local/lib/python3.10/site-packages/torch/nn/modules/module.py:1789, in
    ↪ Module._call_impl(self, *args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or self.
    ↪ _forward_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or _global_backward_hooks
    1788         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

```

```

File ~/.local/lib/python3.10/site-packages/torch/nn/modules/loss.py:834, in
    ↪ BCEWithLogitsLoss.forward(self, input, target)
    832 def forward(self, input: Tensor, target: Tensor) -> Tensor:
    833     """Runs the forward pass."""
--> 834     return F.binary_cross_entropy_with_logits(
    835         input,
    836         target,
    837         self.weight,
    838         pos_weight=self.pos_weight,
    839         reduction=self.reduction,
    840     )

```

```

File ~/.local/lib/python3.10/site-packages/torch/nn/functional.py:3641, in
    ↪ binary_cross_entropy_with_logits(input, target, weight, size_average, reduce,
    ↪ reduction, pos_weight)
    3638     reduction_enum = _Reduction.get_enum(reduction)
    3640 if not (target.size() == input.size()):
-> 3641     raise ValueError(

```

```
3642         f"Target size ({target.size()}) must be the same as input size,␣  
↪({input.size()})"  
3643     )  
3645 return torch.binary_cross_entropy_with_logits(  
3646     input, target, weight, pos_weight, reduction_enum  
3647 )
```

```
ValueError: Target size (torch.Size([1])) must be the same as input size (torch  
↪Size([]))
```